3 Hello, World

3.0.1 What you should know

Once you've read and mastered this chapter, you should know how to edit programs in a text editor or IDLE, save them to the hard disk, and run them once they have been saved.

3.0.2 Printing

Programming tutorials since the beginning of time have started with a little program called "Hello, World!"[1] So here it is:

```
print("Hello, World!")
```

If you are using the command line to run programs then type it in with a text editor, save it as `hello.py` and run it with `python3 hello.py`

Otherwise go into IDLE, create a new window, and create the program as in section Creating and Running Programs[2].

When you run this program, the output will look as follows:

```
Hello, World!
```

Now I'm not going to tell you this every time, but when I show you a program, I recommend that you type it yourself and run it (as opposed to copy/pasting it). I tend to learn and internalize the learning material better when I type it in, and you will probably too!

Now here is a more complicated program:

```
print("Jack and Jill went up a hill")
print("to fetch a pail of water;")
print("Jack fell down, and broke his crown,")
print("and Jill came tumbling after.")
```

When you run this program it prints out:

```
Jack and Jill went up a hill
to fetch a pail of water;
```

1 Here ^{https://en.wikibooks.org/wiki/Computer%20Programming%2FHello%20world} is a great list of the famous "Hello, world!" program in many programming languages. Just so you know how simple Python can be...

2 Chapter 2.0.4 on page 9

```
Jack fell down, and broke his crown,
and Jill came tumbling after.
```

When the computer runs this program it first sees the line:

```
print("Jack and Jill went up a hill")
```

so the computer prints:

```
Jack and Jill went up a hill
```

Then the computer goes down to the next line and sees:

```
print("to fetch a pail of water;")
```

So the computer prints to the screen:

```
to fetch a pail of water;
```

The computer keeps looking at each line, follows the command and then goes on to the next line. The computer keeps running commands until it reaches the end of the program.

Terminology

Now is probably a good time to give you a bit of an explanation of what is happening - and a little bit of programming terminology.

What we were doing above was using a *function* called `print`. The function's name - `print` - is followed by parentheses containing zero or more *arguments*. So in this example

```
print("Hello, World!")
```

there is one *argument*, which is `"Hello, World!"`. Note that this argument is a group of characters enclosed in double quotes (""). This is commonly referred to as a *string of characters*, or *string*, for short. Another example of a string is `"Jack and Jill went up a hill"`. The combination of a function and parentheses with the arguments is a *function call*.

A function and its arguments are one type of *statement* that python has, so

```
print("Hello, World!")
```

is an example of a statement. Basically, you can think of a statement as a single line in a program.

That's probably more than enough terminology for now.

\n in Printing

`\n`, or newline in printing makes the strings after the \n in a new line, it is also an escape character, here is an example:

```
print("Hello, World!\nWhat should I do?")
```

Here is the output:

```
Hello, World!
What should I do?
```

It can be used to put a bunch of strings that are supposed to be on different lines into 1 print statement instead of making multiple print statements

The print statement also sort of uses `\n` even if you do not use it for example:

```
print("Hello, World!")
```

is actually

```
print("Hello, World!\n")
```

Well, there is a difference if you do it manually, but python actually adds a newline "behind the scenes" at the end of the string

3.0.3 Expressions

Here is another program:

```
print("2 + 2 is", 2 + 2)
print("3 * 4 is", 3 * 4)
print("100 - 1 is", 100 - 1)
print("(33 + 2) / 5 + 11.5 is", (33 + 2) / 5 + 11.5)
```

And here is the *output* when the program is run:

```
2 + 2 is 4
3 * 4 is 12
100 - 1 is 99
(33 + 2) / 5 + 11.5 is 18.5
```

As you can see, Python can turn your thousand-dollar computer into a five-dollar calculator.

Arithmetic expressions

In this example, the print function is followed by two arguments, with each of the arguments separated by a comma. So with the first line of the program

```
print("2 + 2 is", 2 + 2)
```

The first argument is the string `"2 + 2 is"` and the second argument is the *arithmetic expression* `2 + 2`, which is one kind of *expression.*

What is important to note is that a string is printed as is (without the enclosing double quotes), but an *expression* is *evaluated*, or converted to its actual value.

Python has seven basic operations for numbers:

Operation	Symbol	Example
Power (exponentiation)	`**`	`5 ** 2 == 25`
Multiplication	`*`	`2 * 3 == 6`
Division	`/`	`14 / 3 == 4.666666666666667`
Integer Division	`//`	`14 // 3 == 4`
Remainder (modulo)	`%`	`14 % 3 == 2`
Addition	`+`	`1 + 2 == 3`
Subtraction	`-`	`4 - 3 == 1`

Notice that there are two ways to do division, one that returns the repeating decimal, and the other that can get the remainder and the whole number. The order of operations is the same as in math:

- parentheses `()`
- exponents `**`
- multiplication `*`, division `/`, integer division `//`, and remainder `%`
- addition `+` and subtraction `-`

So use parentheses to structure your formulas when needed.

3.0.4 Commenting in Python

Often in programming, you are doing something complicated and may not in the future remember what you did. When this happens the program should probably be commented. A *comment* is a note to you and other programmers explaining what is happening. For example:

```
# Not quite PI, but a credible simulation
print(22 / 7)
```

Which outputs

```
3.14285714286
```

Notice that the comment starts with a hash: `#`. Comments are used to communicate with others who read the program and your future self to make clear what is complicated.

Note that any text can follow comment and that when the program is run, the text after the `#` through to the end of that line is ignored. The `#` does not have to be at the beginning of a new line:

```
# Output PI on the screen
print(22 / 7) # Well, just a good approximation
```

3.0.5 Examples

Each chapter (eventually) will contain examples of the programming features introduced in the chapter. You should at least look over them and see if you understand them. If you don't, you may want to type them in and see what happens. Mess around with them, change them and see what happens.

Denmark.py

```
print("Something's rotten in the state of Denmark.")
print("                -- Shakespeare")
```

Output:

```
Something's rotten in the state of Denmark.
                -- Shakespeare
```

School.py

```
# This is not quite true outside of USA
# and is based on my dim memories of my younger years
print("Firstish Grade")
print("1 + 1 =", 1 + 1)
print("2 + 4 =", 2 + 4)
print("5 - 2 =", 5 - 2)
print()
print("Thirdish Grade")
print("243 - 23 =", 243 - 23)
print("12 * 4 =", 12 * 4)
print("12 / 3 =", 12 / 3)
print("13 / 3 =", 13 // 3, "R", 13 % 3)
print()
print("Junior High")
print("123.56 - 62.12 =", 123.56 - 62.12)
print("(4 + 3) * 2 =", (4 + 3) * 2)
print("4 + 3 * 2 =", 4 + 3 * 2)
print("3 ** 2 =", 3 ** 2)
```

Output:

```
Firstish Grade
1 + 1 = 2
2 + 4 = 6
5 - 2 = 3

Thirdish Grade
243 - 23 = 220
12 * 4 = 48
12 / 3 = 4
13 / 3 = 4 R 1

Junior High
123.56 - 62.12 = 61.44
(4 + 3) * 2 = 14
4 + 3 * 2 = 10
3 ** 2 = 9
```

3.0.6 Exercises

1. Write a program that prints your full name and your birthday as separate strings.
2. Write a program that shows the use of all 7 arithmetic operations[3].

Solution

1. Write a program that prints your full name and your birthday as separate strings.

```
print("Ada Lovelace", "born on", "November 27, 1852")
```

```
print("Albert Einstein", "born on", "14 March 1879")
```

```
print(("John Smith"), ("born on"), ("14 March 1879"))
```

Solution

2. Write a program that shows the use of all 7 arithmetic operations[4].

```
print("5**5 = ", 5**5)
print("6*7 = ", 6*7)
print("56/8 = ", 56/8)
print("14//6 = ", 14//6)
print("14%6 = ", 14%6)
print("5+6 = ", 5+6)
print("9-0 = ", 9-0)
```

Footnotes

ca:Python 3 per a no programadors/Hola, món[5]

3 Chapter 3.0.3 on page 15

4 Chapter 3.0.3 on page 15

5 https://ca.wikibooks.org/wiki/Python%203%20per%20a%20no%20programadors%2FHola%2C%20m%C3%B3n

4 Who Goes There?

4.0.1 Input and Variables

Now I feel it is time for a really complicated program. Here it is:

```
print("Halt!")
user_input = input("Who goes there? ")
print("You may pass,",  user_input)
```

When **I** ran it, here is what **my** screen showed:

```
Halt!
Who goes there? Josh
You may pass, Josh
```

Note: After running the code by pressing F5, the python shell will only give output:

```
Halt!
Who goes there?
```

You need to enter your name in the python shell, and then press enter for the rest of the output.

Of course when you run the program your screen will look different because of the `input()` statement. When you ran the program you probably noticed (you did run the program, right?) how you had to type in your name and then press Enter. Then the program printed out some more text and also your name. This is an example of *input*. The program reaches a certain point and then waits for the user to input some data that the program can use later.

Of course, getting information from the user would be useless if we didn't have anywhere to put that information and this is where variables come in. In the previous program `user_input` is a *variable*. Variables are like a box that can store some piece of data. Here is a program to show examples of variables:

```
a = 123.4
b23 = 'Spam'
first_name = "Bill"
b = 432
c = a + b
print("a + b is",c)
print("first_name is",first_name)
print("Sorted Parts, After Midnight or",b23)
```

And here is the output:

```
a + b is 555.4
first_name is Bill
Sorted Parts, After Midnight or Spam
```

Variables store data. The variables in the above program are `a`, `b23`, `first_name`, `b`, and `c`. The two basic types are *strings* and *numbers*. Strings are a sequence of letters, numbers and other characters. In this example `b23` and `first_name` are variables that are storing strings. `Spam`, `Bill`, `a + b is`, `first_name is`, and `Sorted Parts, After Midnight or` are the strings in this program. The characters are surrounded by `"` or `'`. The other type of variables are numbers. Remember that variables are used to store a value, they do not use quotation marks (" and '). If you want to use an actual *value*, you *must* use quotation marks.

```
value1 == Pim
value2 == "Pim"
```

Both look the same, but in the first one Python checks if the value stored in the variable `value1` is the same as the value stored in the *variable* `Pim`. In the second one, Python checks if the string (the actual letters `P`,`i`, and `m`) are the same as in `value2` (continue this tutorial for more explanation about strings and about the `==`).

4.0.2 Assignment

Okay, so we have these boxes called variables and also data that can go into the variable. The computer will see a line like `first_name = "Bill"` and it reads it as "Put the string `Bill` into the box (or variable) `first_name`". Later on it sees the statement `c = a + b` and it reads it as "put the sum of `a + b` or `123.4 + 432` which equals `555.4` into `c`". The right hand side of the statement (`a + b`) is *evaluated* and the result is stored in the variable on the left hand side (`c`). This is called *assignment*, and you should not confuse the assignment equal sign (`=`) with "equality" in a mathematical sense here (that's what `==` will be used for later).

Here is another example of variable usage:

```
a = 1
print(a)
a = a + 1
print(a)
a = a * 2
print(a)
```

And of course here is the output:

```
1
2
4
```

Even if the same variable appears on both sides of the equals sign (e.g., spam = spam), the computer still reads it as, "First find out the data to store and then find out where the data goes."

One more program before I end this chapter:

```
number = float(input("Type in a number: "))
integer = int(input("Type in an integer: "))
text = input("Type in a string: ")
print("number =", number)
print("number is a", type(number))
print("number * 2 =", number * 2)
print("integer =", integer)
print("integer is a", type(integer))
print("integer * 2 =", integer * 2)
print("text =", text)
print("text is a", type(text))
print("text * 2 =", text * 2)
```

The output I got was:

```
Type in a number: 12.34
Type in an integer: -3
Type in a string: Hello
number = 12.34
number is a <class 'float'>
number * 2 = 24.68
integer = -3
integer is a <class 'int'>
integer * 2 = -6
text = Hello
text is a <class 'str'>
text * 2 = HelloHello
```

Notice that `number` was created with `float(input())` ,`int(input())` returns an integer, a number with no decimal point, while `text` created with `input()` returns a string(can be written as `str(input())`, too). When you want the user to type in a decimal use `float(input())`, if you want the user to type in an integer use `int(input())`, but if you want the user to type in a string use `input()`.

The second half of the program uses the `type()` function which tells what kind a variable is. Numbers are of type `int` or `float`, which are short for *integer* and *floating point* (mostly used for decimal numbers), respectively. Text strings are of type `str`, short for *string*. Integers and floats can be worked on by mathematical functions, strings cannot. Notice how when python multiplies a number by an integer the expected thing happens. However when a string is multiplied by an integer the result is that multiple copies of the string are produced (i.e., `text * 2 = HelloHello`).

Operations with strings do different things than operations with numbers. As well, some operations only work with numbers (both integers and floating point numbers) and will give an error if a string is used. Here are some interactive mode examples to show that some more.

```
>>> print("This" + " " + "is" + " joined.")
This is joined.
>>> print("Ha, " * 5)
Ha, Ha, Ha, Ha, Ha,
>>> print("Ha, " * 5 + "ha!")
```

```
Ha, Ha, Ha, Ha, Ha, ha!
>>> print(3 - 1)
2
>>> print("3" - "1")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for -: 'str' and 'str'
>>>
```

Here is the list of some string operations:

Operation	Symbol	Example
Repetition	*	"i" * 5 == "iiiii"
Concatenation	+	"Hello, " + "World!" == "Hello, World!"

4.0.3 Examples

Rate_times.py

```
# This program calculates rate and distance problems
print("Input a rate and a distance")
rate = float(input("Rate: "))
distance = float(input("Distance: "))
time=(distance/ rate)
print("Time:", time)
```

Sample runs:

```
Input a rate and a distance
Rate: 5
Distance: 10
Time: 2.0
```

```
Input a rate and a distance
Rate: 3.52
Distance: 45.6
Time: 12.9545454545
```

Area.py

```
# This program calculates the perimeter and area of a rectangle
print("Calculate information about a rectangle")
length = float(input("Length: "))
width = float(input("Width: "))
Perimeter=(2 * length + 2 * width)
print("Area:", length * width)
print("Perimeter:",Perimeter)
```

Sample runs:

```
Calculate information about a rectangle
Length: 4
Width: 3
Area: 12.0
Perimeter: 14.0
```

```
Calculate information about a rectangle
Length: 2.53
Width: 5.2
Area: 13.156
Perimeter: 15.46
```

Temperature.py

```
# This program converts Fahrenheit to Celsius
fahr_temp = float(input("Fahrenheit temperature: "))
celc_temp = (fahr_temp - 32.0) *( 5.0 / 9.0)
print("Celsius temperature:", celc_temp)
```

Sample runs:

```
Fahrenheit temperature: 32
Celsius temperature: 0.0
```

```
Fahrenheit temperature: -40
Celsius temperature: -40.0
```

```
Fahrenheit temperature: 212
Celsius temperature: 100.0
```

```
Fahrenheit temperature: 98.6
Celsius temperature: 37.0
```

4.0.4 Exercises

Write a program that gets 2 string variables and 2 number variables from the user, concatenates (joins them together with no spaces) and displays the strings, then multiplies the two numbers on a new line.

Solution

Write a program that gets 2 string variables and 2 number variables from the user, concatenates (joins them together with no spaces) and displays the strings, then multiplies the two numbers on a new line.

```
string1 = input('String 1: ')
string2 = input('String 2: ')
float1 = float(input('Number 1: '))
float2 = float(input('Number 2: '))
print(string1 + string2)
print(float1 * float2)
```

ca:Python 3 per a no programadors/Qui hi ha?[1]

1 https://ca.wikibooks.org/wiki/Python%203%20per%20a%20no%20programadors%2FQui%20hi%20ha%3F

5 Count to 10

5.0.1 While loops

Presenting our first *control structure*. Ordinarily the computer starts with the first line and then goes down from there. Control structures change the order that statements are executed or decide if a certain statement will be run. Here's the source for a program that uses the while control structure:

```
a = 0            # FIRST, set the initial value of the variable a to 0(zero).
while a < 10:    # While the value of the variable a is less than 10 do the
 following:
    a = a + 1    # Increase the value of the variable a by 1, as in: a = a + 1!
    print(a)     # Print to screen what the present value of the variable a is.
                 # REPEAT! until the value of the variable a is equal to 9!? See
 note.

                 # NOTE:
                 # The value of the variable a will increase by 1
                 # with each repeat, or loop of the 'while statement BLOCK'.
                 # e.g. a = 1 then a = 2 then a = 3 etc. until a = 9 then...
                 # the code will finish adding 1 to a (now a = 10), printing the

                 # result, and then exiting the 'while statement BLOCK'.
                 #              --
                 # While a < 10: |
                 #     a = a + 1 |<--[ The while statement BLOCK ]
                 #     print (a) |
                 #               --
```

And here is the extremely exciting output:

```
1
2
3
4
5
6
7
8
9
10
```

(And you thought it couldn't get any worse after turning your computer into a five-dollar calculator?)

So what does the program do? First it sees the line `a = 0` and sets `a` to zero. Then it sees `while a < 10:` and so the computer checks to see if `a < 10`. The first time the computer sees this statement, `a` is zero, so it is less than 10. In other words, as long as `a` is less than ten, the computer will run the tabbed in statements. This eventually makes `a` equal to

ten (by adding one to `a` again and again) and the `while a < 10` is not true any longer. Reaching that point, the program will stop running the indented lines.

Always remember to put a colon "**:**" at the end of the `while` statement line!

Here is another example of the use of `while`:

```
a = 1
s = 0
print('Enter Numbers to add to the sum.')
print('Enter 0 to quit.')
while a != 0:
    print('Current Sum:', s)
    a = float(input('Number? '))
    s = s + a
print('Total Sum =', s)
```

```
Enter Numbers to add to the sum.
Enter 0 to quit.
Current Sum: 0
Number? 200
Current Sum: 200.0
Number? -15.25
Current Sum: 184.75
Number? -151.85
Current Sum: 32.9
Number? 10.00
Current Sum: 42.9
Number? 0
Total Sum = 42.9
```

Notice how `print('Total Sum =', s)` is only run at the end. The `while` statement only affects the lines that are indented with whitespace. The `!=` means does not equal so `while a != 0:` means as long as `a` is not zero run the tabbed statements that follow.

Note that `a` is a floating point number, and not all floating point numbers can be accurately represented, so using `!=` on them can sometimes not work. Try typing in 1.1 in interactive mode.

Infinite loops or Never Ending Loop

Now that we have while loops, it is possible to have programs that run forever. An easy way to do this is to write a program like this:

```
while 1 == 1:
   print("Help, I'm stuck in a loop.")
```

The "`==`" operator is used to test equality of the expressions on the two sides of the operator, just as "`<`" was used for "less than" before (you will get a complete list of all comparison operators in the next chapter).

This program will output `Help, I'm stuck in a loop.` until the heat death of the universe or you stop it, because 1 will forever be equal to 1. The way to stop it is to hit the Control (or *Ctrl*) button and *C* (the letter) at the same time. This will kill the program. (Note: sometimes you will have to hit enter after the Control-C.) On some systems, nothing will stop it, short of killing the process–so avoid!

5.0.2 Examples

Fibonacci sequence

Fibonacci-method1.py

```
# This program calculates the Fibonacci sequence
a = 0
b = 1
count = 0
max_count = 20

while count < max_count:
    count = count + 1
    print(a, end=" ")  # Notice the magic end=" " in the print function
 arguments
                       # that keeps it from creating a new line.
    old_a = a    # we need to keep track of a since we change it.
    a = b
    b = old_a + b
print()  # gets a new (empty) line.
```

Output:

```
 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
```

Note that the output is on a single line because of the extra argument `end=" "` in the `print` arguments.

Fibonacci-method2.py

```
# Simplified and faster method to calculate the Fibonacci sequence
a = 0
b = 1
count = 0
max_count = 10

while count < max_count:
    count = count + 1
    print(a, b, end=" ")  # Notice the magic end=" "
    a = a + b
    b = a + b
print()  # gets a new (empty) line.
```

Output:

```
 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
```

Fibonacci-method3.py

```
a = 0
b = 1
count = 0
maxcount = 20

#once loop is started we stay in it
while count < maxcount:
    count += 1
    olda = a
    a = a + b
```

```
    b = olda
    print(olda,end=" ")
print()
```

Output:

```
 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
```

Enter password

Password.py

```
# Waits until a password has been entered.  Use Control-C to break out without
# the password

#Note that this must not be the password so that the
# while loop runs at least once.
password = str()

# note that != means not equal
while password != "unicorn":
    password = input("Password: ")
print("Welcome in")
```

Sample run:

```
Password: auo
Password: y22
Password: password
Password: open sesame
Password: unicorn
Welcome in
```

5.0.3 Exercises

Write a program that asks the user for a Login Name and password. Then when they type ”lock”, they need to type in their name and password to unlock the program.

Solution

Write a program that asks the user for a Login Name and password. Then when they type ”lock”, they need to type in their name and password to unlock the program.

```
name = input("What is your UserName: ")
password = input("What is your Password: ")
print("To lock your computer type lock.")
command = None
input1 = None
input2 = None
while command != "lock":
    command = input("What is your command: ")
while input1 != name:
```